Smart Asset Management and Resource Allocation Platform

Design Document — Cult Open 2026

IIT Roorkee Cultural Council Open Projects

TEAM MEMBERS

Aditya Raj

Enrollment No. 2413009

Tanmay Jain

Enrollment No. 24112106

Gaurav Kumar Meena

Enrollment No. 24113045

Document Type	System Design & Architecture Document
Submission For	Cult Open Projects 2026
Problem Statement	Smart Asset Management and Resource Allocation Platform
Max Pages	8 Pages

TABLE OF CONTENTS

Contents

1. Problem Understanding 3
2. System Architecture 3
2.1. High-Level Architecture 3
2.2. Client Tier: Frontend Application 3
2.3. Application Tier: Backend API 4
2.4. Data Tier: PostgreSQL and Supabase 4
3. Database Schema 4
3.1. Core Tables 4
4. Entity Relationship Diagram (ERD) 5
5. API Overview 6
5.1. Authentication Module 6
5.2. Assets Module 6
5.3. Bookings Module 6
5.4. Queue Module 6
6. Design Decisions 7
6.1. Separation of Models and Units 7
6.2. The Conflict Engine vs. Simple Counters 7
6.3. Waitlist Isolation and Expiry 7
6.4. Reliability Scoring Algorithm 8
6.5. Supabase as a PaaS 8

SECTION 1

Problem Understanding

In modern organizational environments — from university clubs and corporate IT departments to community makerspaces — the management of physical assets remains a critical operational challenge. Organizations frequently deal with shared resources such as DSLR cameras, studio lighting equipment, audio systems, costumes, stage props, recording equipment, and event infrastructure that multiple stakeholders require access to simultaneously.

The Cultural Council of IIT Roorkee exemplifies this challenge. Resource management currently relies on fragmented communication channels, spreadsheets, manual registers, and informal coordination processes. This leads to several recurring failure modes:

- **Double-Bookings and Scheduling Conflicts:** Without an automated reservation engine, two users may inadvertently be promised the same resource, causing operational friction and delayed projects.
- **Lack of Accountability:** When an item is returned damaged or not returned at all, identifying the responsible party is difficult; historical usage data is absent or tedious to compile.
- **Inefficient Queue Management:** 'First-come, first-served' on a shared spreadsheet is frustrating and perceived as unfair; there is no structured waitlist mechanism.
- **Maintenance Blind Spots:** Tracking the lifecycle and health of individual units is nearly impossible; preventative maintenance is ignored and damage reports are lost.
- **Administrative Overhead:** Administrators spend excessive time manually auditing inventory and tracking overdue items instead of focusing on strategic tasks.

The Smart Asset Management Platform addresses these issues by providing a robust, automated, and user-friendly web application. It centralises the booking engine, enforces waitlist fairness, generates unique QR codes for instant physical tracking, and calculates user reliability scores — transforming asset management from a reactive, error-prone chore into a proactive, data-driven process.

SECTION 2

System Architecture

The platform follows a modern decoupled client-server model. This separation of concerns ensures the backend and frontend scale independently and can easily integrate with future mobile applications or third-party services.

2.1 High-Level Architecture

The system consists of three primary tiers: the **Client Tier** (Frontend Dashboard), the **Application Tier** (Backend API), and the **Data Tier** (Database and Storage). Each tier communicates over well-defined interfaces, enabling independent scaling and deployment.

Client Tier	Application Tier	Data Tier
Next.js / React SPA Axios HTTP Client JWT Token Management QR Code Scanner	FastAPI (Python) Auth · Asset · Booking Queue · QR · Maintenance Services	PostgreSQL (Supabase) SQLAlchemy ORM Alembic Migrations Supabase Storage (S3)

Table 1: Three-tier architecture overview

2.2 Client Tier: Frontend Application

The frontend is a Single-Page Application (SPA) built with **Next.js** and **React.js**. Next.js provides robust routing, server-side rendering for SEO, and optimised asset delivery. The client communicates with the backend exclusively via RESTful JSON APIs using the **Axios** HTTP client. It handles presentation logic, form validation,

JWT token management (including automatic refresh workflows), and provides an interface for the embedded QR code scanner.

2.3 Application Tier: Backend API

The backend is built with **FastAPI**, a high-performance Python 3.8+ framework chosen for its speed (comparable to Node.js and Go), automatic interactive API documentation (Swagger/ReDoc), and native asynchronous programming support. The architecture is modular, divided into distinct feature domains:

Service	Responsibility
Auth Service	User registration, login, JWT issuance, Argon2 password hashing, RBAC.
Asset Service	CRUD operations for Asset Models and Asset Units; image upload to external storage.
Booking Service	Core conflict engine; validates date ranges; manages booking lifecycle.
Queue Service	Waitlist management; triggers next user with exclusive booking window on return.
QR Service	Generates dynamic QR codes; processes scan payloads for instant diagnostic data.
Maintenance Service	Handles damage reports, opens tickets, and logs condition transitions.

Table 2: Backend service modules

2.4 Data Tier: PostgreSQL and Supabase

The application relies on **PostgreSQL** for relational data storage, hosted via **Supabase**. PostgreSQL provides the ACID guarantees required by a booking engine where transactional integrity is paramount. **SQLAlchemy** serves as the ORM and **Alembic** handles schema migrations. For unstructured data (asset images, return photos, QR code PNGs), the system integrates with **Supabase Storage**, acting as an S3-compatible object storage layer.

SECTION 3

Database Schema

The schema is highly normalised to eliminate redundancy and maintain data integrity. It features comprehensive foreign key constraints and PostgreSQL Enums to enforce state validity throughout the asset lifecycle.

3.1 Core Tables

Table	Key Columns & Notes
Users	id (UUID), email, password_hash, full_name, role (Enum: user / pending_admin / admin), reliability_score (Float, default 100), is_email_verified (Boolean).
Asset Models	id, name, category, description, location, purchase_date, status, created_at. Represents a generic asset type (e.g., 'MacBook Pro M2').
Asset Units	id, asset_model_id (FK), unit_code, serial_number, condition (Enum), status (Enum: Available / Issued / Maintenance / Retired), qr_code_url. Represents an individual physical item.
Bookings	booking_id, user_id (FK), asset_model_id (FK), preferred_unit_id (FK), quantity, purpose, start_date, end_date, status (Enum: Pending / Approved / Issued / Returned / Overdue / Cancelled / Rejected).
Booking Unit Assignments	id, booking_id (FK), asset_unit_id (FK), condition_before, condition_after, return_remarks, assigned_at, returned_at. Junction table resolving Bookings ↔ AssetUnits.

Asset Queue	id, asset_model_id, user_id, position, status (Waiting / Reserved / Expired / Completed), reserved_at, expires_at.
Maintenance Requests	id, asset_unit_id, issue, priority, status, reported_by, assigned_to.
Audit Logs	id, user_id, action, entity_type, entity_id, timestamp, metadata. Immutable system-wide event log.

Table 3: Core database tables

The deliberate separation between *Asset Models* and *Asset Units* is a key design decision: a user books an 'Asset Model' (they want *a* camera); the admin approves and then issues specific 'Asset Units' (Camera #12 and Camera #14) at pickup. This allows flexibility while enforcing granular physical tracking.

SECTION 4

Entity Relationship Diagram

The ERD below maps the foreign key constraints and cardinalities between all core domain models. One-to-many (1:N) relationships are the dominant pattern; the BookingUnitAssignments table resolves the many-to-many relationship between Bookings and physical AssetUnits.

Entity A	Card.	Entity B	Relationship Description
Users	**1 : N**	**Bookings**	A user creates many bookings
Users	**1 : N**	**AssetQueue**	A user can hold multiple queue positions
Users	**1 : N**	**MaintenanceRequests**	A user reports multiple maintenance issues
Users	**1 : N**	**AuditLogs**	Every user action is logged
AssetModels	**1 : N**	**AssetUnits**	A model has many physical units
AssetModels	**1 : N**	**Bookings**	A model can have many bookings
AssetModels	**1 : N**	**AssetQueue**	A model has its own waitlist
AssetUnits	**1 : N**	**BookingUnitAssignments**	A unit is assigned across many bookings
AssetUnits	**1 : N**	**MaintenanceRequests**	A unit can have many maintenance tickets
Bookings	**1 : N**	**BookingUnitAssignments**	One booking ↔ multiple physical units
Bookings	**1 : N**	**ReturnPhotos**	Return photos attached to a booking

Table 4: Entity Relationship summary

SECTION 5

API Overview

The backend exposes a versioned RESTful JSON API. All endpoints — except public authentication routes — require a valid JWT Access Token in the **Authorization: Bearer <token>** header. The API strictly follows HTTP verb semantics: GET (retrieval), POST (creation), PUT/PATCH (modification), DELETE (removal). The base path for all routes is **/api/v1/**.

5.1 Authentication Module (/api/v1/auth)

Method	Endpoint	Description
POST	`/login`	Accepts email/password; returns access + refresh JWTs.
POST	`/register`	Creates a new user account; triggers OTP verification email.
POST	`/refresh`	Exchanges a valid refresh token for a new access token.

5.2 Assets Module (/api/v1/assets)

Method	Endpoint	Description
GET	`/`	Search and filter asset models with pagination support.

POST	/	[Admin] Create a new asset model.
GET	/{model_id}/units	Retrieve all physical units for a specific model.
GET	/scan/{unit_code}	Retrieve comprehensive unit data via QR scan (status, holder, maintenance).

5.3 Bookings Module (/api/v1/bookings)

Method	Endpoint	Description
POST	/	Request a booking; conflict engine evaluates date ranges before accepting.
PUT	/{booking_id}/approve	[Admin] Approve a pending booking.
PUT	/{booking_id}/issue	[Admin] Assign specific AssetUnits to an approved booking.
PUT	/{booking_id}/return	[Admin] Process return; evaluate condition; adjust reliability score.

5.4 Queue Module (/api/v1/queue)

Method	Endpoint	Description
POST	/join/{model_id}	Add the current user to the waitlist for an asset model.
GET	/{model_id}	View current queue length and per-user status.

SECTION 6

Design Decisions

6.1 Separation of Asset Models and Units

A naive implementation might use a simple 'quantity' column on an asset table. However, this prevents tracking individual item health — it would be impossible to determine which specific laptop was returned with a cracked screen. By separating generic *Asset Models* from physical *Asset Units*, the platform achieves the best of both worlds: users book 'a laptop' without needing serial numbers, while administrators maintain strict physical tracking with unique QR codes for every device.

6.2 Conflict Engine vs. Simple Counters

Rather than relying on a simple 'items currently checked out' counter, the platform implements a robust **Conflict Engine** that validates date overlaps. The engine iterates over the requested date range, calculates the theoretical sum of all reserved units on any given day, and compares it against total inventory and active waitlist reservations. This mathematical approach guarantees zero double-bookings and supports future-dated reservations accurately.

6.3 Waitlist Isolation and Expiry

When an asset is returned, broadcasting a mass notification creates a frustrating race condition. Instead, the system uses an **isolation model**: the first person in the queue receives an exclusive 4-hour reservation window. The backend actively shields that unit from all other users during this period. If the timer expires, an asynchronous worker marks the reservation as Expired, applies a reliability score penalty for holding up the queue, and passes the reservation to the next person.

6.4 Reliability Scoring Algorithm

A gamified **Reliability Score** automates accountability without requiring administrators to remember which users are untrustworthy. The system applies objective deductions — for example, -1 point per day overdue. If a user's score drops below 50, they are algorithmically blocked from joining waitlists or booking high-value assets. This eliminates administrative confrontation and relies on consistent, transparent, systemic enforcement.

6.5 Supabase as a Platform-as-a-Service

Supabase was selected to host PostgreSQL and object storage. By leveraging a PaaS, the team eliminated DevOps overhead for database replication, backup scheduling, and S3 IAM policies. Supabase's integrated PgBouncer connection pooling is particularly valuable for FastAPI, which can easily overwhelm standard PostgreSQL connection limits under high asynchronous load.